f6c06b9d-c3c3-42d5-90b8-dfa6fb9ae5ec

<current>

<request>

```
You are working on an issue in the 'justinlietz93/Cogito'
repository.

<agent_instructions>
## Instructions

## Personality Profile
- **Intellect:** Analytical
- **Rigor:** Rigorous
- **Objectivity:** Objective
- **Detail-Orientation:** Meticulous
- **Autonomy:** Autonomous
- **Formality:** Formal
- **Utility:** Utilitarian

## Response Output Requirements
Generates highly structured, comprehensive analytical reports
summarizing all details, organized logically (e.g., first coarse
grain, more fine grained as commensurate), providing clear
description and sequence for each point.
Prioritizes absoluteness, comprehensiveness, and reviews items from
multiple angles before making a conclusion.
Outputs favor highly structured, hierarchically itemized reports,
often categorized by process, system, section, function,
relationships, etc... Uses clinical labeling.
Communication is minimal but information-rich, concise, dense,
incisive, technically correct, unambiguous, and uses precise
methodological/statistical terminology.

#### Recursive Hierarchical Decomposition & Step Validation (Apex
Rigorous Synthesis Protocol Applied)
  Employs structured decomposition (Goal: Critique Artifact -> Phase
-> Task -> Step/Check) for planning the review process (e.g.,
Phases: Context Understanding, Methodological Analysis,
Architectural Dissection,
  System Components, Soundness Evaluation, Overview Synthesis).
```

```
  Each defined Step (e.g., "Verify function X exists and is used for
specified process Y," "Average the pipeline robustness at each
codebase level junction," "Verify the documentation accurately
reflects the code,"
  "Quantify number of orphaned code findings.") undergoes mandatory
internal validation before being finalized:
  - Self-Critique: Checks that the review was holistic as well as
narrowed, that all items were addressed, and that the review is
accurate.
#### Microscopic Precision & Detail
  Operates with extreme attention to detail in identifying subtle
flaws in logic, methodology, statistical application, data
interpretation, and textual consistency.

### Interaction Style
#### Clinical & Objective
  Communication is purely functional, analytical, using precise
methodological and statistical terminology. Focuses entirely on the
artifact, the criteria, the evidence, and the logical assessment.

### Exclusions (What it Does NOT Do)
  - Does not engage in any non-functional interaction or express
subjective opinions/preferences.
  - Does not role-play beyond this functional RC-Apex persona.
  - Does not provide a critique until the artifact and review
criteria are fully analyzed and a critique plan is validated.
  - Does not ask for clarification unless internal resolution fails
for a critical ambiguity preventing objective assessment. Makes no
assumptions about author intent or unstated context.
  - Does not compromise on objectivity, methodological rigor,
statistical validity, or evidence-based justification.
  - All points are specific and substantiated.
</agent_instructions>

<repository_context>
I've cloned the repository in the directory /home/runner/work/
Cogito/Cogito (not in /tmp/inputs). Always use absolute paths when
referring to files in the repository.
</repository_context>

<current_datetime>2025-11-23T01:00:24.439Z</current_datetime>

Consider the following problem statement:

  <problem_statement>
  High-level proposal (will refine once you confirm backend
  details).
  I’ll need some specifics about Cogito’s backend to tailor this
```

precisely:
1. What protocols are exposed? (REST, GraphQL, WebSockets, gRPC via proxy?)
2. How does authentication/authorization work? (OAuth2/OIDC, JWT, API keys, sessions?)
3. What are the core feature domains? (e.g., user management, analytics, knowledge graphs, task orchestration, data ingestion, reporting?)
4. Do any features require real-time updates (notifications, streaming metrics)?
5. Any multi-tenant concerns?
6. Expected scale (concurrent users, data volume) and performance constraints.
7. Accessibility / compliance targets (WCAG level? Internationalization?)
8. Preferred tech stack (React, Vue, Svelte, Next.js, Remix, etc.) or constraints.
9. Do you anticipate third-party plugins or internal-only extensibility?
10. CI/CD environment and hosting (Vercel, Cloudflare Pages, self-hosted, Docker behind Nginx, etc.)
Let me know these and I'll tailor concrete module boundaries, data contracts, and example code scaffolds.

Below is a comprehensive, extensible front-end architecture you can adopt now, with placeholders that will snap to your backend once details arrive.

Architecture Goals
- Full feature coverage of current backend
- Rapid addition of new backend capabilities without refactoring core
- Support horizontal expansion (new feature domains) and vertical expansion (deeper functionality per domain)
- Strong separation of concerns: UI, data access, state, cross-cutting services
- Pluggable feature modules loaded dynamically (code-splitting)
- Schema or contract-driven UI generation where possible
- Consistent design system + theming
- Observability, testability, and resilience baked in

Recommended Technology Stack (adjustable)
- Framework: Next.js (for SSR/SSG, routing, performance) or Remix (if you want enhanced data conventions). If purely SPA, use Vite + React.
- Language: TypeScript everywhere.

```
- State: Domain-scoped stores using Redux Toolkit or Zustand
(Redux better for predictable enterprise patterns; Zustand
simpler; can mix).
- Data Layer:
  - If REST: TanStack Query for caching, invalidation,
optimistic updates.
  - If GraphQL: Apollo Client or urql with codegen (schema-
driven types).
  - Real-time: Socket.io, native WebSocket wrapper, or GraphQL
subscriptions provider.
- Styling: Tailwind + Radix UI + CSS Variables for design tokens
OR a fully custom component library with Storybook.
- Forms: React Hook Form + Zod (schema validation). If GraphQL,
auto-generate form defaults from schema.
- Build: Turborepo if multi-package (design-system, core-
services, feature modules).
- Testing: Vitest/Jest (unit), Playwright/Cypress (e2e),
Storybook interaction tests.
- Quality: ESLint (custom rules per layer), TypeScript strict,
Lighthouse budget in CI.

-------------------------------------------------
Project Monorepo Structure (example)
/apps
  /web                (Next.js front-end shell)
  /admin              (Optional separate app if needed)
/packages
  /design-system      (Shared UI components, tokens, icons)
  /core-services      (Auth, API client factories, logging,
error mapping)
  /feature-user       (User domain: pages, components, hooks,
slices)
  /feature-analytics  (Analytics domain)
  /feature-content    (Content/domain-specific logic)
  /feature-realtime   (Event stream visualizations)
/configs
  eslint/, tsconfig/, jest/, storybook/
/scripts
  codegen, build, lint, release

Each feature package exports:
- Routes (metadata)
- Navigation items
- Store slices / hooks
- Service adapters
- Component registry
- Permissions manifest
```

```
---------------------------------------------------
Modular Feature Pattern
Define a FeatureManifest interface:
interface FeatureManifest {
  id: string;
  version: string;
  routes: RouteDef[];
  navItems?: NavItem[];
  permissions?: PermissionDef[];
  register?(ctx: AppContext): void;
  unload?(): void;
}

On app boot:
1. Load static feature manifests (core).
2. Optionally fetch dynamic manifest index from backend (to
allow backend-driven UI enabling).
3. Dynamically import feature bundles (code splitting).
4. Register their routes, nav, permissions, and inject store
slices.

Vertical Expansion
- Each feature declares capability descriptors (e.g., analytics
supports dashboards, alerts, exports).
- Backend advertises new capabilities; manifest expands; front-
end UI either:
  - Generates new panels via schema (e.g., new metric type =
auto-card)
  - Or provides plugin slot for manual enhancement.

Horizontal Expansion
- Adding a new domain = create /feature-newDomain package
implementing manifest + service adapters; no core changes.

---------------------------------------------------
Data Access Layer
Create a unified service factory:
class ApiClient {
  constructor(private auth: AuthProvider, private config:
ApiConfig) {}
  request<T>(req: ApiRequest): Promise<T> { /* inject tracing,
retry, error normalization */ }
}

Feature modules never call fetch directly; they call domain
services:
```

```
UserService.getProfile()
AnalyticsService.getDashboards()
These wrap TanStack Query hooks:
export function useUserProfile() {
  return useQuery(['user','profile'], () =>
UserService.getProfile(), { staleTime: 60000 });
}

Swappable transport: If you add GraphQL later, implement
GraphQLTransport conforming to Transport interface.

------------------------------------------------
Schema-Driven UI (Optional but powerful)
If backend can return JSON schemas / GraphQL schema
introspection:
- Auto-generate forms, tables, filters.
- Use a component mapper keyed by schema field types.
- New backend attribute → UI appears with minimal code.

Example:
Backend returns:
{
  "entity": "User",
  "fields": [
    {"name":"displayName","type":"string","label":"Display
Name","required":true},
    {"name":"roles","type":"enum","values":
["admin","editor","viewer"]}
  ]
}
Front-end renders dynamic form from schema + validation using
Zod generated from schema.

------------------------------------------------
Auth & Permissions
- AuthProvider abstracts token storage, refresh, session
invalidation.
- Permission engine loads user claims and merges with feature
manifests.
- UI components use <Require permission="analytics.view"> to
gate rendering.
- Navigation dynamically filters unauthorized entries.

------------------------------------------------
Routing
Next.js file-based routes for shell (e.g., /app). Inside /app
use a dynamic router config merged from manifests:
```

```
const dynamicRoutes = aggregateManifests().flatMap(m =>
m.routes)
Render via <ManifestRouter routes={dynamicRoutes} />

--------------------------------------------------
Design System & Theming
- tokens.json: color scales, spacing, typography, animation.
- Provide dark/light themes; theme switch stored in
localStorage; accessible contrast ratios.
- Components must be composition-friendly (e.g., <DataTable
columns={...} data={...} /> with plugin column injectors).
- Use Storybook for each component with accessibility and
interaction tests.

--------------------------------------------------
State Strategy
- Local ephemeral UI state: React useState / useReducer.
- Domain data: Query cache.
- Cross-feature events: EventBus (simple observable or RxJS).
Example events:
  "user.updated", "analytics.alertRaised".
- Persist minimal UI preferences in localStorage with versioning
(e.g., layout config).

--------------------------------------------------
Real-Time Layer (if needed)
Feature-realtime package exposes a subscribe(eventType, handler)
API that internally uses WebSockets or SSE.
Example:
Realtime.subscribe('analytics.metric.update', updateMetricCache)
Provide hooks: useRealtimeChannel(channelName) returning live
data with graceful degradation (falls back to polling if socket
fails).

--------------------------------------------------
Error Handling & Observability
- Global ErrorBoundary per major layout region.
- Normalization function maps HTTP/gRPC/GraphQL errors to
unified AppError.
- Logging service dispatches structured logs to console in dev
and to external collector (if integrated).
- Performance monitoring: web-vitals + custom marks (feature
load times).
- Tracing IDs correlate front-end requests with backend spans
(inject trace header).

--------------------------------------------------
```

```
Internationalization (i18n) (if needed)
- Message catalogs per feature package.
- Keys composed as: feature.user.profile.title
- Lazy-load locale bundles.

------------------------------------------------
Accessibility
- Automated checks (axe-core in Storybook, CI audits).
- Keyboard navigation support, focus outlines, semantic markup.
- Announce dynamic content (ARIA live regions for real-time
events).

------------------------------------------------
Testing Strategy
Unit: Services, utility functions (Vitest).
Component: Storybook interaction tests + RTL render tests.
Integration: Mock backend with MSW (Mock Service Worker).
E2E: Playwright against deployed preview env.
Contract: If using OpenAPI/GraphQL, generate types and run drift
detection (compare local spec vs backend live).

------------------------------------------------
Continuous Delivery
- Feature packages versioned independently? (Optional; can treat
all as single version.)
- Automated codegen (OpenAPI / GraphQL) before build.
- Build matrix: lint → typecheck → test → e2e → deploy preview →
promote.
- Feature flag system (LaunchDarkly or simple backend-based
flags) to dark-launch UI modules.

------------------------------------------------
Example Feature Skeleton (feature-user)
/packages/feature-user/src/manifest.ts
export const userManifest: FeatureManifest = {
  id: 'user',
  version: '1.0.0',
  routes: [
    { path: '/users', component: UsersPage, permission:
'user.list' },
    { path: '/users/:id', component: UserDetailPage, permission:
'user.view' }
  ],
  permissions: [
    { key: 'user.list', description: 'List users' },
    { key: 'user.view', description: 'View user details' }
  ],
```

```
  register(ctx) {
    ctx.store.injectSlice('user', userSlice)
  }
}

Service Example:
export const UserService = {
  async getProfile(): Promise<User> {
    return apiClient.request<User>({ method: 'GET', url: '/api/
users/me' })
  }
}

Hook Example:
export function useMyProfile() {
  return useQuery(['user','me'], UserService.getProfile)
}

-------------------------------------------------
Plugin System (Optional for external extensibility)
Expose a global CogitoPlugin API:
interface CogitoPlugin {
  id: string;
  provides: { routes?: RouteDef[]; components?: Record<string,
React.ComponentType>; }
  activate(ctx: PluginContext): void;
  deactivate?(): void;
}
Plugins discovered by:
1. Backend returning plugin manifest list.
2. Front-end fetches list, dynamic import via module federation
or remote entry script.
Security: sandbox via iframe for untrusted external plugins
(postMessage bridge).

-------------------------------------------------
Roadmap Phases
Phase 1: Core shell (routing, auth, design system, API client,
user feature)
Phase 2: Additional features (analytics, content) + manifest
loader
Phase 3: Real-time layer + event bus
Phase 4: Schema-driven forms/tables
Phase 5: Plugin system & dynamic remote modules
Phase 6: Observability hardening & performance budgets
Phase 7: Internationalization & accessibility audits
```

```
------------------------------------------------
Deliverables Once Backend Details Provided
- Tailored manifest definitions for each existing backend
feature
- Concrete API client adapters (REST vs GraphQL)
- Example codegen config (OpenAPI / GraphQL)
- Domain event map
- Initial folder scaffold commands
- CI config draft (GitHub Actions YAML)
- Sample Storybook stories

Next Step: Please share the backend feature list + API style +
auth method so I can convert this into a precise implementation
plan and provide scaffold code.
</problem_statement>
```

Implement the necessary changes to the repository so that the requirements specified in the <problem_statement> are met.

Ensure that you lint, build and test early and iteratively to validate each code change thoroughly.